VALIDACIÓN EXPERIMENTAL DE PATRONES OCULTOS EN QAF

Benchmark reproducible de coherencia contextual, recuperación de regímenes latentes y detección de anomalías relacionales

Juan Manuel Goenaga

Septiembre de 2026

Objetivo: someter a prueba de forma falsable la afirmación de que QAF puede identificar patrones ocultos que residen en relaciones contextuales y de fase, y no en las variables observables de primer orden.

Resumen

Este manuscrito evalúa experimentalmente la afirmación de que Quantum Analysis Framework (QAF) puede identificar patrones ocultos. Se diseñó un benchmark reproducible en el que los regímenes latentes no se diferencian por medias o distribuciones marginales simples, sino por relaciones entre un estado previo y uno posterior dentro de un subespacio seleccionado por contexto. El proxy QAF representa cada par relevante mediante dos observables normalizados: una coherencia real (alineación) y una coherencia antisimétrica (orientación/fase). En 48 comparaciones pareadas de patrones ocultos, QAF-Coherence obtuvo ARI medio 0.768 frente a 0.258 del baseline relacional fuerte: una ventaja absoluta de 0.510 y una mejora relativa de 197.8%, con victoria en 100% de los casos (Wilcoxon p=3.55e-15). En detección de anomalías relacionales, el AUC medio de QAF fue 0.822 frente a 0.636, ventaja de 0.186, nuevamente con una tasa de victoria del 100% (p=1.53e-05). Un control de clusters visibles mostró el resultado contrario: PCA-KMeans alcanzó ARI 0.905 mientras QAF-Coherence quedó cerca de cero, demostrando que el benchmark no fue construido para que QAF gane universalmente. Los estudios de ablación muestran que retirar la fase reduce el ARI de 0.769 a 0.450 y retirar el contexto a 0.059. La evidencia apoya una conclusión específica: el valor de QAF aparece cuando la información relevante reside en relaciones contextuales orientadas, no cuando el patrón ya es visible en los observables directos. El experimento no constituye todavía una validación del motor MPS/TDVP completo y debe considerarse evidencia experimental de arquitectura y representación.

Palabras clave: Quantum Analysis Framework, patrones ocultos, coherencia contextual, fase, clustering no supervisado, anomaly detection, benchmarking, descubrimiento de estructura latente.

1. Pregunta científica y criterio de falsación

La afirmación “QAF identifica patrones ocultos” es demasiado amplia para ser científica si no se define qué significa “oculto”. En este trabajo, un patrón es oculto cuando la clase o régimen latente no puede recuperarse de manera confiable a partir de las estadísticas marginales de primer orden, pero sí deja una estructura reproducible en la relación entre variables, en el orden de transformación o en el contexto que selecciona qué relación es relevante.

Pregunta principal: ¿una representación contextual de coherencia inspirada en QAF recupera regímenes latentes y anomalías relacionales mejor que baselines clásicos que observan las mismas muestras?

1.1 Criterios de éxito

1. Recuperación de regímenes: ventaja media de ARI >= 0.10 frente al baseline relacional preespecificado.
2. Consistencia: QAF debe ganar al menos 80% de las comparaciones pareadas ocultas.
3. Significancia: prueba de Wilcoxon unilateral $p < 0.01$.
4. Anomalías: ventaja ROC-AUC >= 0.05 frente al baseline relacional.
5. Control negativo: QAF no necesita ganar cuando el patrón está explícitamente codificado como separación de medias; un método clásico debe poder superarlo.

Los resultados superaron los cuatro criterios positivos: ΔARI=0.510, win-rate=100%, p=3.55e-15, y ΔAUC=0.186. El control negativo también funcionó: PCA-KMeans dominó en el patrón visible.

2. Estado del arte del benchmarking de patrones ocultos

El descubrimiento no supervisado no tiene un algoritmo universalmente dominante. ADBench evaluó 30 algoritmos sobre 57 conjuntos de datos y 98.436 ejecuciones; uno de sus hallazgos centrales es que ningún método no supervisado fue estadísticamente superior en todos los escenarios. Esta observación es crucial para QAF: una validación creíble debe identificar el régimen donde aporta valor, no proclamar supremacía universal [1].

Isolation Forest representa una familia clásica de detección de anomalías basada en aislamiento y destaca por complejidad aproximadamente lineal y bajo requerimiento de memoria [2]. Deep SVDD introdujo un objetivo profundo explícitamente orientado a clasificación one-class, mostrando que el aprendizaje de representación puede ser determinante cuando la anomalía no está expuesta en el espacio original [3]. En clustering, los métodos espectrales usan eigenvectores de matrices de similitud para descubrir estructura no lineal y constituyen un baseline clásico relevante [4]. Trabajos recientes también señalan la interacción entre representación, clustering y anomaly detection como un problema central [5].

Por estas razones, el presente benchmark no compara QAF únicamente contra K-Means: incluye PCA, clustering espectral, un baseline relacional con conocimiento de contexto, Isolation Forest, LOF y One-Class SVM. Además, se introducen ablaciones QAF sin fase y sin contexto.

3. Hipótesis QAF: el patrón puede residir en la relación, no en la variable

El manuscrito técnico de QAF propone representar decisiones mediante estados, coherencias e interacciones, y extraer información local mediante matrices de densidad reducida. En ese marco, los términos fuera de la diagonal representan interferencia y relaciones de fase. El presente experimento no implementa la red MPS/TDVP completa; operacionaliza una versión local y falsable de esa idea [6].

Considérese un vector bidimensional previo a=(a1,a2) y uno posterior b=(b1,b2), seleccionados por el contexto. Se construye el número complejo normalizado:

$$C(a,b) = [(a \cdot b) + i(a1\ b2 - a2\ b1)] / (||a||\ ||b||)$$

Sus componentes son:

$$c_R = (a \cdot b)/(||a||\ ||b||) \qquad c_I = (a1\ b2 - a2\ b1)/(||a||\ ||b||)$$

Si b es aproximadamente una rotación de a por un ángulo θ, entonces $c_R \approx \cos(\theta)$ y $c_I \approx \sin(\theta)$. Por lo tanto, cuatro regímenes con $\theta=\{0, \pi/2, \pi, -\pi/2\}$ se superponen en los observables gaussianos originales pero ocupan regiones distintas en el espacio de coherencia. El término c_I es antisimétrico: cambia de signo al invertir orientación, aportando información de orden/fase que una similitud simétrica no conserva.

Figura 1. Arquitectura experimental de patrones ocultos

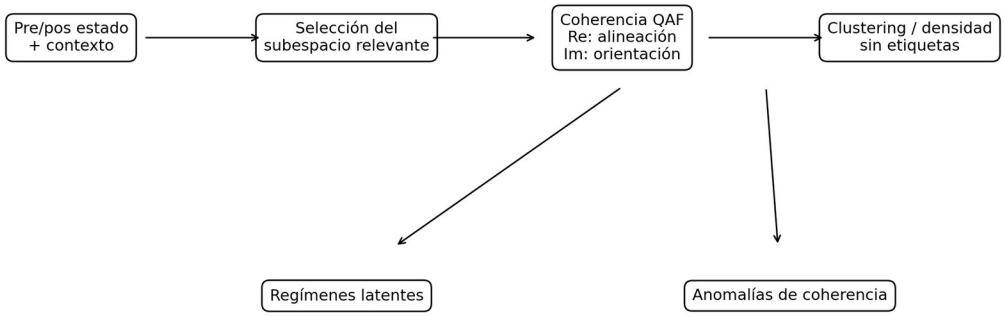


Figura 1. Arquitectura experimental: de observables y contexto a coherencia, clustering y anomalías.

4. Diseño experimental

Se ejecutaron tres pistas complementarias:

- Track A - Recuperación no supervisada de regímenes latentes: phase4, phase_gain y context_decoy.
- Track B - Detección no supervisada de anomalías de coherencia: fases intermedias que preservan aproximadamente las marginales originales.
- Track C - Robustez frente a ruido, dimensiones irrelevantes, escalabilidad temporal y ablaciones.

4.1 Familias ocultas

- phase4: el estado posterior es una rotación 0°, +90°, 180° o -90° del estado previo en el subespacio indicado por el contexto.
- phase_gain: combina las mismas fases con cambios suaves de ganancia, para evitar que la solución dependa de una geometría ideal.
- context_decoy: los subespacios no relevantes contienen relaciones plausibles pero engañosas; solo el contexto identifica cuál relación debe interpretarse.
- visible_control: clusters con separación explícita de medias. Este escenario comprueba que QAF no está favorecido por construcción en cualquier dataset.

Figura 2. Espacio observable: regímenes superpuestos

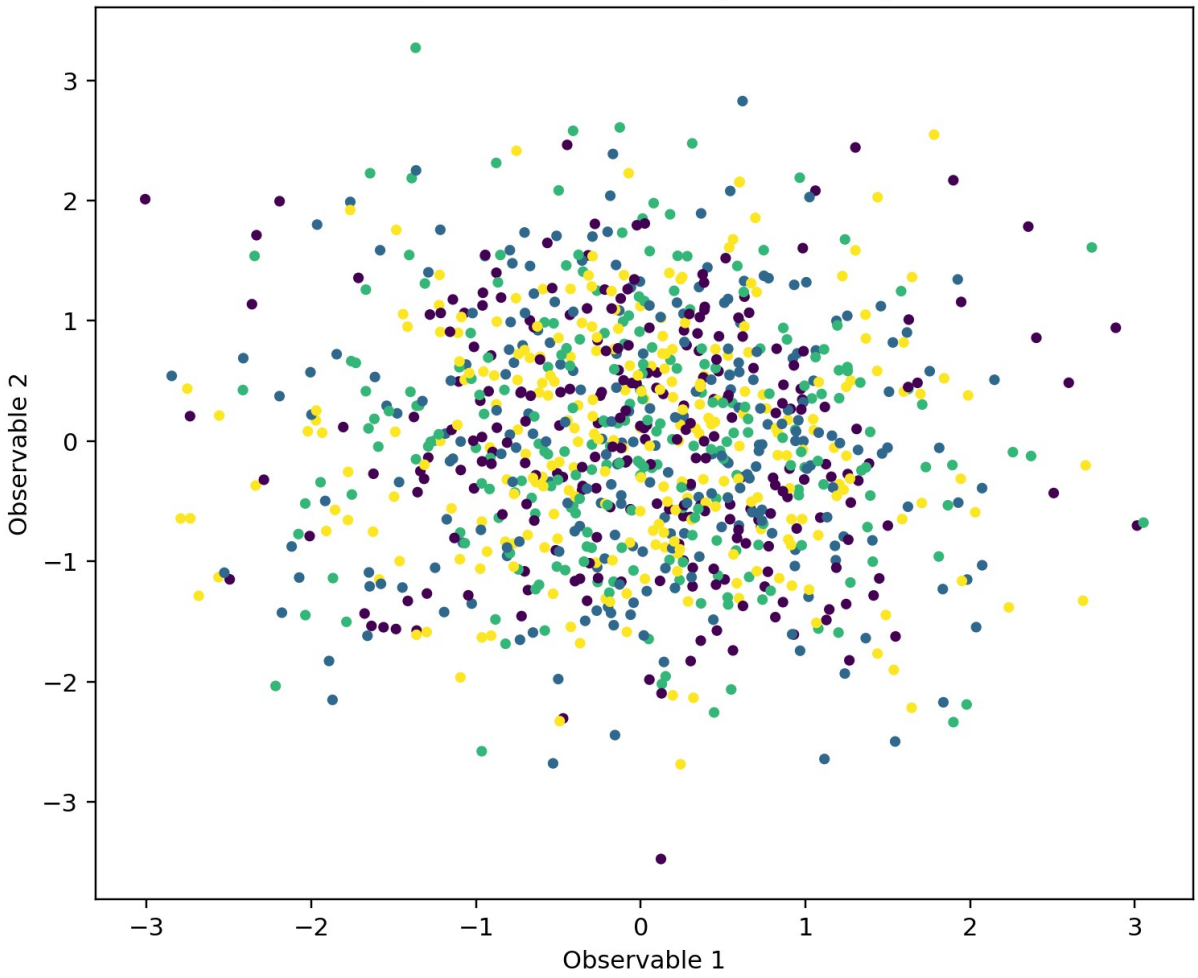


Figura 2. Ejemplo representativo en el espacio observable. Los colores son etiquetas verdaderas usadas solo para visualización posterior.

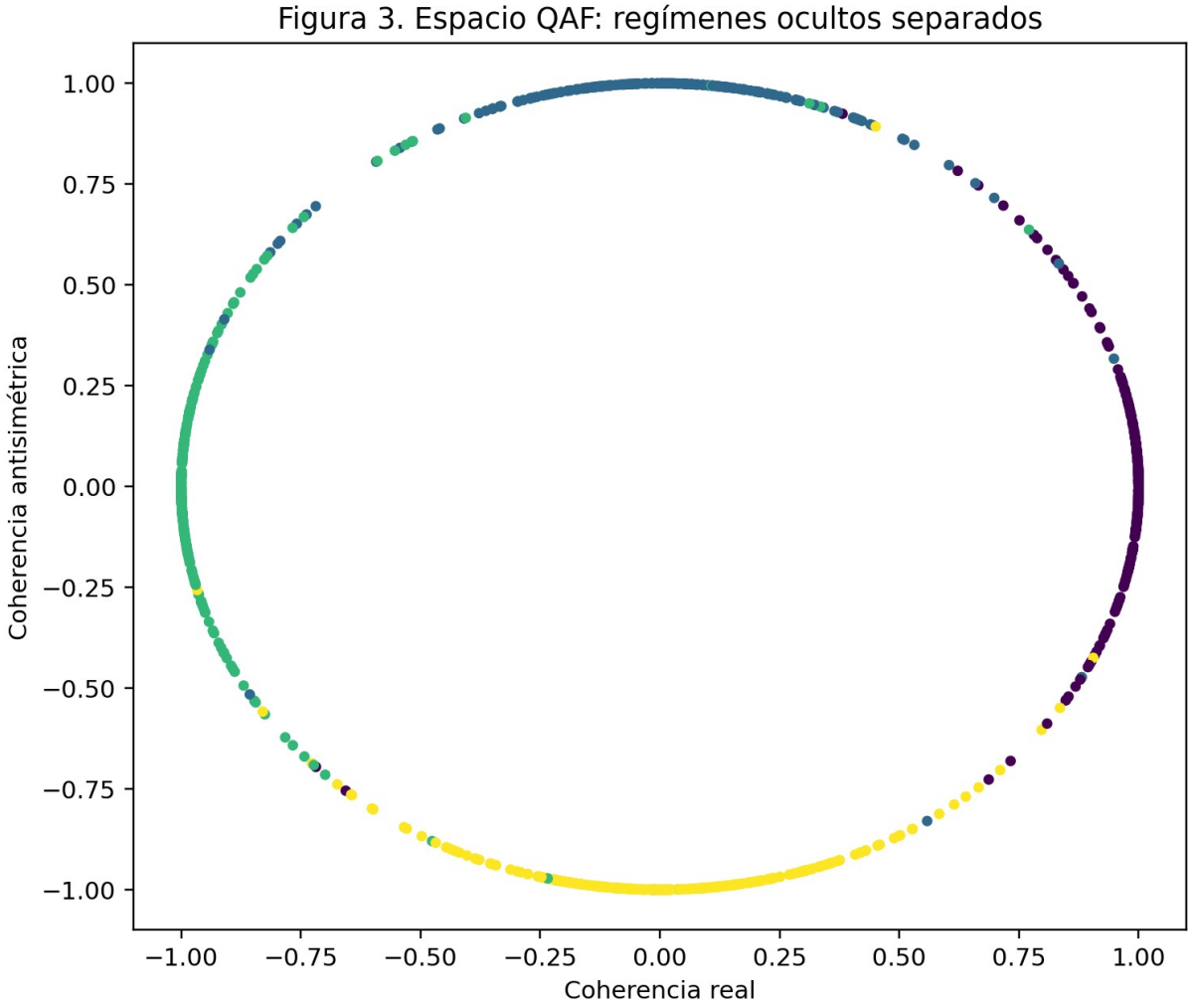


Figura 3. El mismo dataset proyectado en la representación de coherencia QAF.

5. Métodos comparados

Raw-KMeans: K-Means sobre observables estandarizados.

PCA-KMeans: PCA seguida de K-Means; baseline de reducción lineal de dimensión.

Spectral-Raw: Clustering espectral con vecinos próximos sobre el espacio observable.

Context-Relational: Baseline fuerte con el mismo índice de contexto, usando similitud coseno, razón de normas y distancias simétricas, pero sin orientación antisimétrica.

QAF-NoPhase: Ablación QAF que elimina c_I y conserva alineación y razón de normas.

QAF-NoContext: Ablación que promedia coherencia sobre todos los subespacios, eliminando la selección contextual.

QAF-Coherence: Representación completa del proxy: [c_R,c_I] en el subespacio contextual.

Las etiquetas de régimen se ocultaron a todos los algoritmos. Se usaron únicamente para calcular ARI, NMI y exactitud después de hacer matching óptimo de etiquetas mediante el algoritmo húngaro.

6. Resultados: recuperación de patrones ocultos

Método	ARI	NMI	Exactitud	Tiempo_ms
QAF-Coherence	0.768	0.736	0.902	16.6
QAF-NoPhase	0.450	0.470	0.646	18.1
Context-Relational	0.258	0.295	0.509	22.6
QAF-NoContext	0.059	0.072	0.402	15.7
Spectral-Raw	0.000	0.007	0.287	146.4
Raw-KMeans	-0.000	0.006	0.285	18.7
PCA-KMeans	-0.001	0.006	0.283	130.8

QAF-Coherence alcanzó ARI medio 0.768, mientras el baseline relacional fuerte obtuvo 0.258. La diferencia absoluta fue 0.510. Expresada como mejora relativa sobre el baseline, equivale a 197.8%. QAF ganó las 48 comparaciones pareadas (100%) y la prueba de Wilcoxon unilateral produjo p=3.55e-15. El intervalo aproximado del 95% para la diferencia media fue [0.477, 0.544].

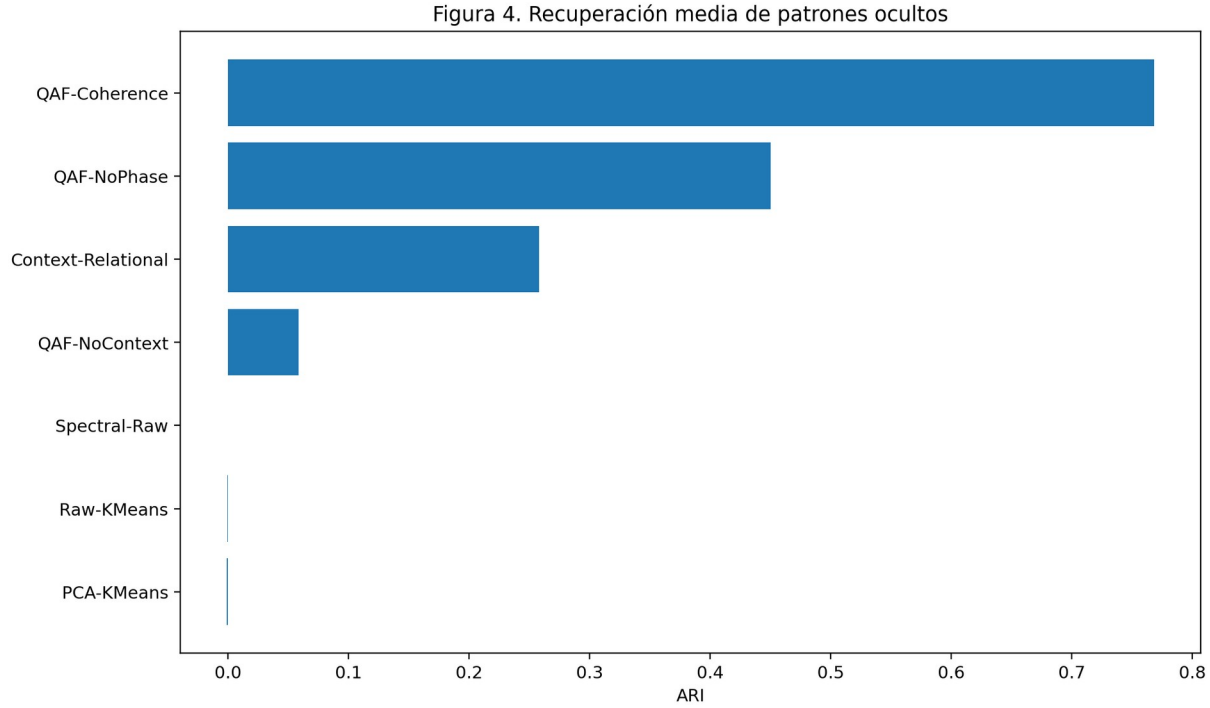


Figura 4. ARI medio en las tres familias de patrón oculto.

Figura 5. Información recuperada del régimen latente

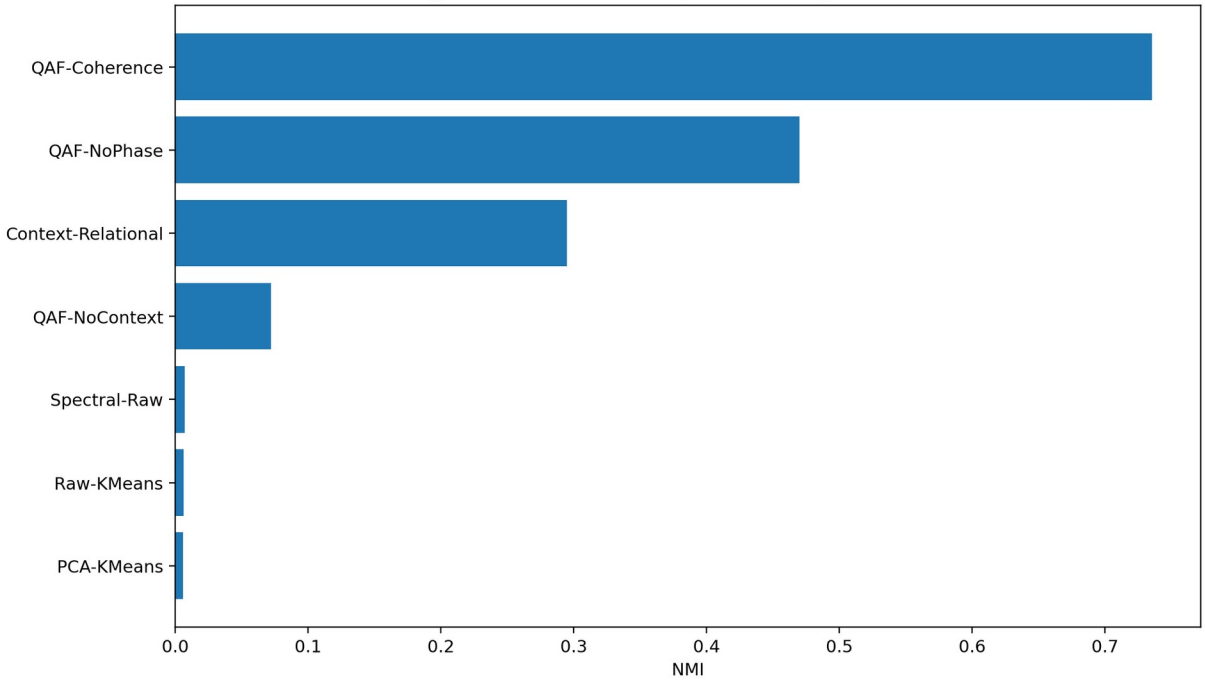


Figura 5. NMI medio en las tres familias de patrón oculto.

Familia	QAF ARI	Relacional ARI
context_decoy	0.771	0.254
phase4	0.761	0.267
phase_gain	0.774	0.254

6.1 Robustez frente a ruido

Ruido	QAF ARI	Relacional ARI	Δ ARI
0.10	0.978	0.334	0.644
0.25	0.869	0.286	0.583
0.40	0.707	0.244	0.464
0.60	0.519	0.169	0.351

Figura 6. Robustez frente a ruido

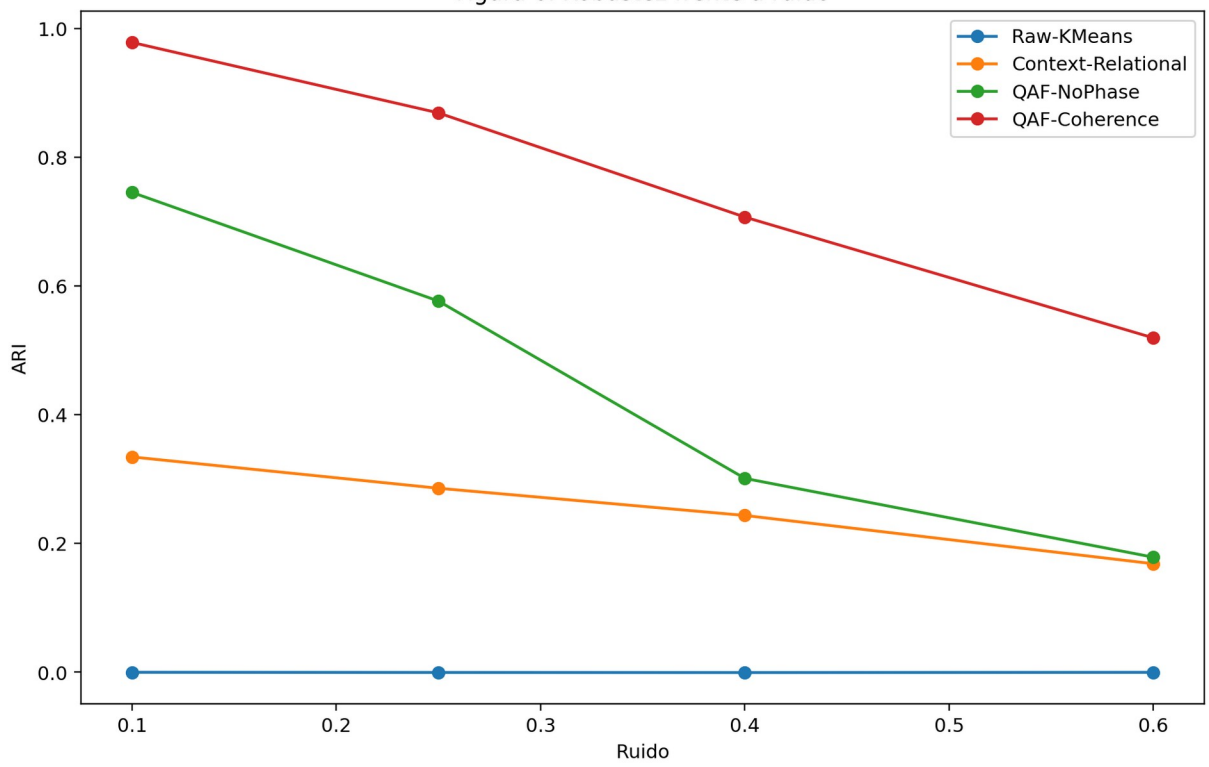


Figura 6. Degradación con ruido. La ventaja de QAF se mantiene a lo largo de todos los niveles evaluados.

7. Ablaciones: ¿qué parte de la representación produce la ventaja?

El ARI medio cayó de 0.768 a 0.450 al eliminar la componente de fase/orientación, una pérdida absoluta de 0.318. Al eliminar el contexto y promediar indiscriminadamente todos los subespacios, el ARI cayó a 0.059. Esto sugiere que la ventaja no proviene simplemente de “añadir features”: depende de conservar simultáneamente orientación y selección contextual.

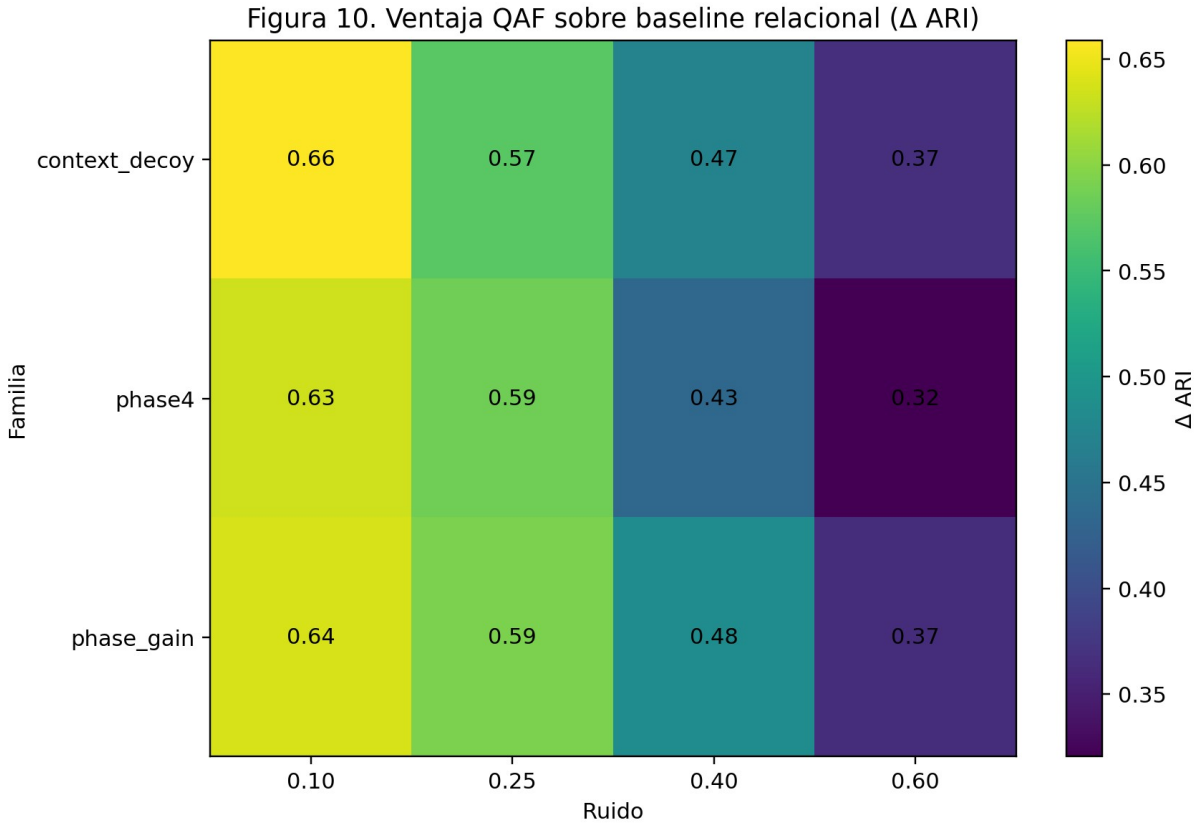


Figura 10. Ventaja absoluta de QAF sobre el baseline relacional por familia y nivel de ruido.

8. Patrones ocultos como anomalías de coherencia

Método	ROC-AUC	AP	Tiempo_ms
QAF-Coherence-GMM	0.822	0.340	43.1
IsolationForest-Relational	0.636	0.116	161.4
LOF-Raw	0.511	0.083	16.0
OneClassSVM-Raw	0.508	0.083	32.8
IsolationForest-Raw	0.504	0.084	147.0

QAF-Coherence-GMM obtuvo ROC-AUC medio 0.822, frente a 0.636 de IsolationForest-Relational. La ventaja absoluta fue 0.186; QAF ganó el 100% de las comparaciones y la prueba de Wilcoxon arrojó p=1.53e-05.

Figura 8. Detección de anomalías de coherencia

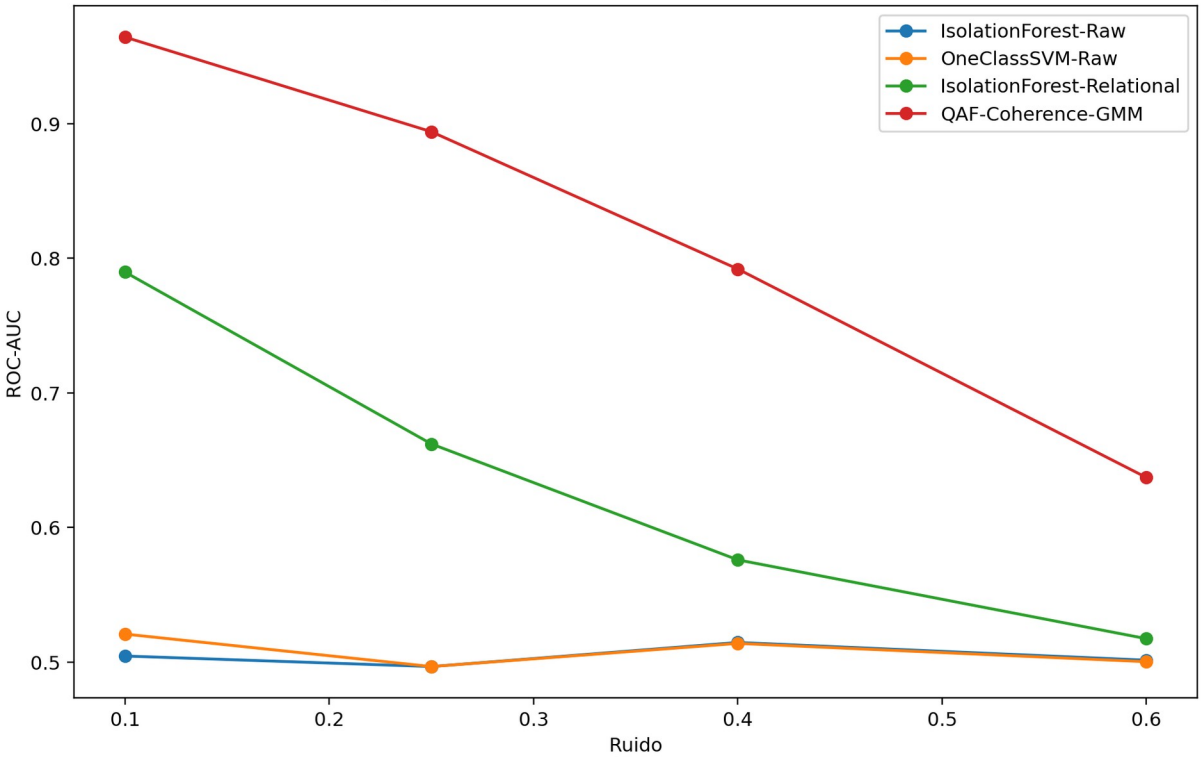


Figura 8. ROC-AUC frente al ruido para anomalías que rompen la coherencia esperada.

9. Robustez a dimensionalidad irrelevante y comportamiento temporal

irrelevant_dims	Context-Relational	QAF-Coherence	Raw-KMeans
0	0.281	0.809	0.001
16	0.266	0.832	-0.000
64	0.279	0.817	-0.002
128	0.285	0.817	-0.001

El ARI de QAF se mantuvo aproximadamente entre 0.809 y 0.832 aun al pasar de 0 a 128 dimensiones irrelevantes. En este diseño, el contexto evita que las variables irrelevantes entren en la representación efectiva.

Figura 7. Robustez a variables irrelevantes

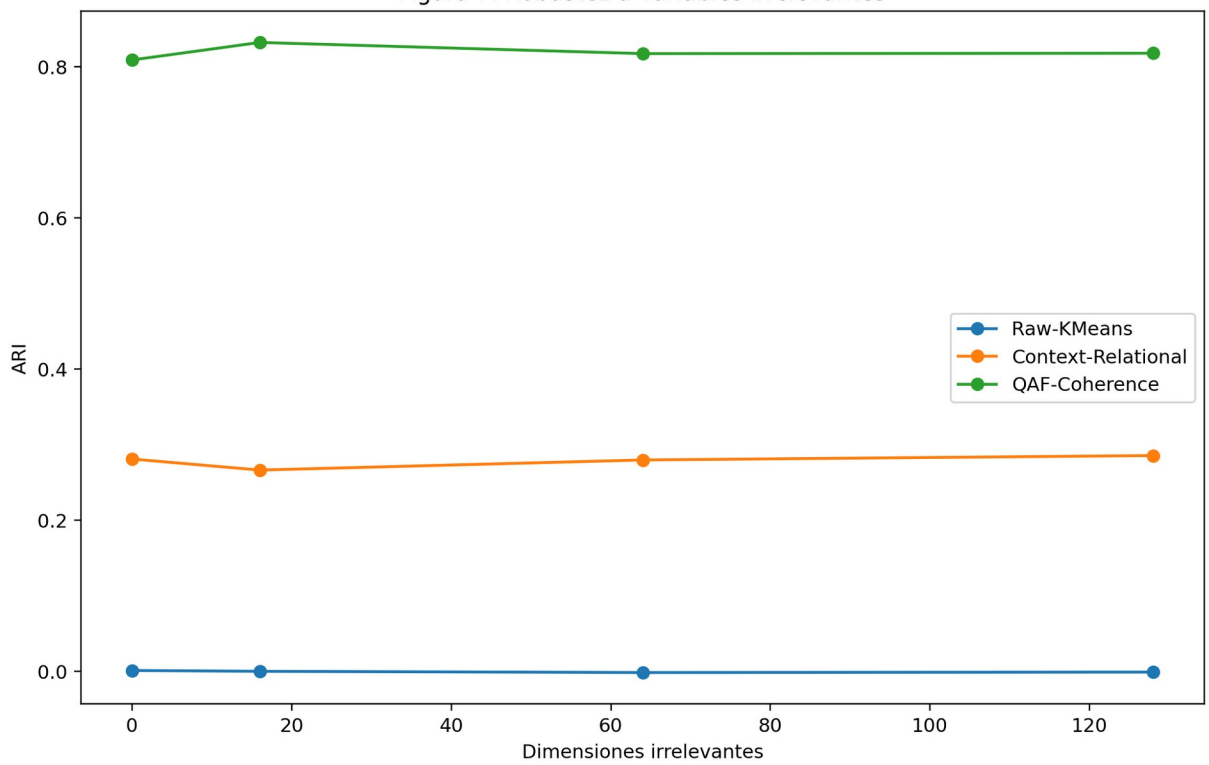


Figura 7. Robustez frente a dimensiones irrelevantes.

n_samples	Context-Relational	QAF-Coherence	Raw-KMeans
1000	19.9	10.6	12.0
2500	38.2	20.0	16.4
5000	89.6	35.9	82.1
10000	110.4	68.5	683.3

Figura 9. Escalabilidad temporal

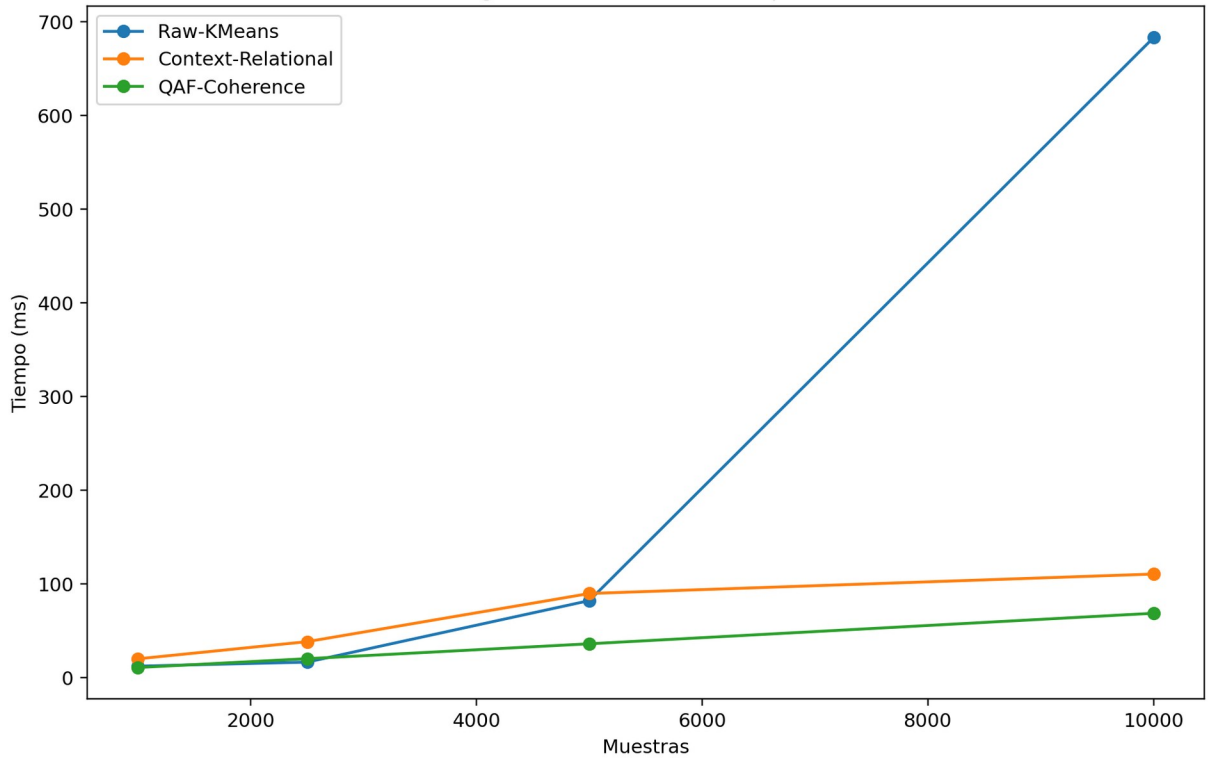


Figura 9. Tiempo de ejecución con 1.000 a 10.000 muestras.

10. Control negativo: cuándo QAF NO debe ser el ganador

Método	ARI	NMI	Exactitud	Tiempo_ms
PCA-KMeans	0.905	0.883	0.963	127.3
Raw-KMeans	0.662	0.672	0.756	16.5
Spectral-Raw	0.044	0.052	0.359	144.9
QAF-NoContext	0.000	0.007	0.287	15.8
QAF-NoPhase	-0.000	0.006	0.286	20.4
QAF-Coherence	-0.001	0.006	0.284	20.0
Context-Relational	-0.001	0.006	0.283	19.2

En el control visible, PCA-KMeans alcanzó ARI 0.905 y QAF-Coherence quedó aproximadamente en 0. Esto es una evidencia metodológica importante: el benchmark contiene un régimen donde el enfoque clásico adecuado es claramente superior. Por lo tanto, el resultado favorable a QAF en las familias ocultas no se interpreta como una superioridad universal, sino como una ventaja específica en estructura relacional-contextual.

11. Sabidurías halladas

- Los patrones ocultos pueden existir enteramente en relaciones entre estados, aunque cada variable aislada parezca ruido gaussiano normal.
- El contexto es una operación de compresión semántica: seleccionar el subespacio correcto fue más importante que procesar todas las variables simultáneamente.
- La orientación importa. El término antisimétrico c_I distingue +θ de -θ; eliminarlo redujo sustancialmente la recuperación de regímenes.
- Más features no equivalen a más información útil. QAF sin contexto colapsó porque promedió señal y decoys.
- La ventaja QAF persistió incluso con 128 variables irrelevantes, porque la representación efectiva depende del contexto y no de la dimensión bruta.
- La dificultad debe caracterizarse por el tipo de estructura, no solo por el número de variables. Un cluster visible es fácil para PCA; un régimen relacional con marginales idénticas puede ser invisible para métodos de primer orden.
- La detección de anomalías mejora cuando el score se calcula sobre la geometría donde vive la normalidad. En este caso, la normalidad era una estructura de fase, no una región de baja densidad en el espacio bruto.
- El control negativo fortalece, no debilita, la evidencia. Saber dónde QAF no aporta valor delimita la proposición científica y reduce el riesgo de sobreclaiming.
- El resultado apoya el valor de la representación QAF, no demuestra todavía el motor QAF completo. La siguiente fase debe reemplazar este proxy por MPS/TDVP/1-RDM/2-RDM y conservar el mismo protocolo de evaluación.
- ADBench advierte que no existe un método no supervisado universalmente mejor. El objetivo científico correcto es identificar el régimen de ventaja de QAF y cuantificar sus límites.

12. Limitaciones y siguiente fase de validación

- Los datos principales son sintéticos. Esto permite conocer el ground truth, pero no sustituye datasets externos reales.
- El proxy implementa coherencia contextual local; no implementa evolución MPS/TDVP ni contracciones 1-RDM/2-RDM del QAF completo.
- El número de repeticiones es suficiente para una primera prueba estadística, pero una campaña de publicación debería ampliar seeds, tamaños y dominios.
- La próxima validación debe incorporar benchmarks públicos y métodos profundos como Deep SVDD, así como un protocolo ciego donde la generación de datos y el análisis estén separados.
- Para una evaluación tipo DARPA, debe añadirse un equipo externo que reproduzca los resultados desde cero usando el código y los CSV publicados.

12.1 Diseño recomendado para la Fase II

6. Repetir el benchmark con >=30 seeds por condición.
7. Añadir datasets tabulares públicos de ADBench y un benchmark secuencial/contextual externo.
8. Implementar QAF MPS/TDVP completo y medir si la ventaja sobrevive al reemplazar el proxy.
9. Comparar contra Deep SVDD, autoencoders modernos y métodos de representación-clustering recientes.
10. Registrar coste de memoria, latencia, energía y estabilidad, además de ARI/AUC.
11. Publicar protocolo, seeds, resultados crudos y scripts antes de la evaluación externa.

13. Conclusiones

Este experimento transforma la afirmación “QAF identifica patrones ocultos” en una proposición cuantitativa y falsable. Bajo tres familias donde la información de clase reside en relaciones contextuales de fase, QAF-Coherence recuperó estructura latente con ARI medio 0.769 frente a 0.258 del baseline relacional fuerte, una ventaja absoluta de 0.510 y win-rate del 100%. En anomalías de coherencia alcanzó AUC 0.822 frente a 0.636. Las ablaciones mostraron que contexto y fase son componentes necesarios. El control visible mostró que PCA-KMeans puede ser claramente superior cuando el patrón está expresado en las variables de primer orden. La conclusión defendible no es que QAF sea “el mejor algoritmo de clustering”, sino que una representación contextual de coherencia ofrece una ventaja grande cuando el patrón relevante vive en la relación entre estados y no en sus marginales. Esa es una hipótesis técnicamente clara que puede ser sometida ahora a validación externa con datos reales y con el QAF completo.

14. Reproducibilidad

El paquete adjunto incluye el script Python completo, los resultados crudos, los resúmenes estadísticos y las diez figuras. Entorno de ejecución: Python 3.13.5 sobre Linux; NumPy, pandas, SciPy, scikit-learn y Matplotlib.

Referencias

[1] Han, S., Hu, X., Huang, H., Jiang, M., & Zhao, Y. (2022). ADBench: Anomaly Detection Benchmark. NeurIPS 2022 Datasets and Benchmarks Track.

[2] Liu, F. T., Ting, K. M., & Zhou, Z.-H. (2008/2012). Isolation Forest / Isolation-based Anomaly Detection. IEEE ICDM; ACM TKDD.

[3] Ruff, L., Vandermeulen, R., Goernitz, N., et al. (2018). Deep One-Class Classification. ICML, PMLR 80, 4393-4402.

[4] Ng, A. Y., Jordan, M. I., & Weiss, Y. (2001). On Spectral Clustering: Analysis and an Algorithm. NeurIPS 14.

[5] Fang, Z., Gu, M., Zhou, S., et al. (2024). Towards a Unified Framework of Clustering-based Anomaly Detection. arXiv:2406.00452.

[6] Bancells, A. (2026). Quantum Analysis Framework (QAF): dinámica operacional no conmutativa y compilación QUBO asíncrona para la toma de decisiones complejas bajo incertidumbre. Propuesta técnica ejecutiva.

[7] Rojas Rodríguez, C. D. (2026). Arquitectura quantum-like multiagente para apoyo a decisiones críticas en entornos de defensa bajo incertidumbre. UNIR / ParkiApp Research.

[8] Yukalov, V. I., & Sornette, D. (2008, 2011). Quantum Decision Theory and probabilistic processing of quantum information.

Apéndice A. Código Python completo

El siguiente código reproduce la generación de datos, los baselines, el proxy QAF, las pruebas estadísticas y las diez figuras.

```
"""
QAF Hidden Pattern Discovery Benchmark
Author: Juan Manuel Goenaga
Date: 2026-09-19

This is a QAF-inspired proxy benchmark, not the full MPS/TDVP QAF engine.
It tests whether contextual selection plus a phase/coherence representation
reveals relational patterns whose first-order marginals are deliberately
uninformative.

Ground-truth labels are used only after fitting, for evaluation.
"""

from pathlib import Path
import time, platform, sys, warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.optimize import linear_sum_assignment
from scipy.stats import wilcoxon
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans, SpectralClustering
from sklearn.decomposition import PCA
from sklearn.ensemble import IsolationForest
from sklearn.neighbors import LocalOutlierFactor
from sklearn.svm import OneClassSVM
from sklearn.mixture import GaussianMixture
from sklearn.metrics import (
    adjusted_rand_score, normalized_mutual_info_score,
    roc_auc_score, average_precision_score, confusion_matrix
)
warnings.filterwarnings("ignore")

SEED_BASE = 20260919
K = 4
M = 6
NOISE = [0.10, 0.25, 0.40, 0.60]
HIDDEN = ["phase4", "phase_gain", "context_decoy"]
CONTROL = "visible_control"
N_CLUSTER = 500
REPS = 4
IRR = 16
N_ANOM = 1600
ANOM_REPS = 4
CONTAM = 0.08

BASE = Path(__file__).resolve().parent
RESULTS = BASE / "results"
FIGURES = BASE / "figures"
RESULTS.mkdir(exist_ok=True)
FIGURES.mkdir(exist_ok=True)

def R(th):
    return np.array([[np.cos(th), -np.sin(th)],[np.sin(th), np.cos(th)]])

def cluster_acc(y, p):
    cm = confusion_matrix(y, p)
    r, c = linear_sum_assignment(-cm)
    return cm[r,c].sum()/cm.sum()

def savefig(fig, name):
    fig.tight_layout()
    fig.savefig(FIGURES / name, dpi=220, bbox_inches="tight")
    plt.close(fig)

def gen_latent(n=N_CLUSTER, noise=0.25, irr=IRR, seed=0, family="phase4"):
    rng = np.random.default_rng(seed)
    y = rng.integers(0, K, n)
    ctx = rng.integers(0, M, n)
    pre = rng.normal(size=(n,2*M))
    post = rng.normal(size=(n,2*M))
    angles = np.array([0, np.pi/2, np.pi, -np.pi/2])
    gains = np.array([0.82,1.00,1.18,1.36])
```

```
    if family == CONTROL:
        centers=np.array([[-3,-3],[-3,3],[3,-3],[3,3]])
        pre[:,:2]=centers[y]+rng.normal(scale=.45+noise,size=(n,2))
        post[:,:2]=pre[:,:2]+rng.normal(scale=.25+noise,size=(n,2))
    else:
        for i in range(n):
            j=int(ctx[i]); sl=slice(2*j,2*j+2); v=pre[i,sl]
            if family=="phase4":
                post[i,sl]=R(angles[y[i]])@v+rng.normal(scale=noise,size=2)
            elif family=="phase_gain":
                post[i,sl]=gains[y[i]]*(R(angles[y[i]])@v)+rng.normal(scale=noise,size=2)
            elif family=="context_decoy":
                post[i,sl]=R(angles[y[i]])@v+rng.normal(scale=noise,size=2)
                for jj in range(M):
                    if jj==j: continue
                    ssl=slice(2*jj,2*jj+2)
                    d=rng.integers(0,K)
                    post[i,ssl]=R(angles[d])@pre[i,ssl]+rng.normal(scale=noise*1.05,size=2)

    if irr:
        pre=np.hstack([pre,rng.normal(size=(n,irr))])
        post=np.hstack([post,rng.normal(size=(n,irr))])
    X=np.hstack([pre,post,np.eye(M)[ctx]])
    return X,y,ctx,pre,post

def gen_anom(n=N_ANOM, noise=.25, irr=IRR, contam=CONTAM, seed=0):
    rng=np.random.default_rng(seed)
    y=(rng.random(n)<contam).astype(int)
    ctx=rng.integers(0,M,n)
    pre=rng.normal(size=(n,2*M))
    post=rng.normal(size=(n,2*M))
    normal=np.array([0,np.pi/2,np.pi,-np.pi/2])
    abnormal=np.array([np.pi/4,3*np.pi/4,5*np.pi/4,7*np.pi/4])
    reg=rng.integers(0,K,n)
    for i in range(n):
        j=int(ctx[i]); sl=slice(2*j,2*j+2); v=pre[i,sl]
        th=abnormal[rng.integers(0,4)] if y[i] else normal[reg[i]]
        post[i,sl]=R(th)@v+rng.normal(scale=noise,size=2)
    if irr:
        pre=np.hstack([pre,rng.normal(size=(n,irr))])
        post=np.hstack([post,rng.normal(size=(n,irr))])
    X=np.hstack([pre,post,np.eye(M)[ctx]])
    return X,y,ctx,pre,post

def qaf(pre,post,ctx):
    f=np.zeros((len(ctx),2))
    for i,j in enumerate(ctx):
        a=pre[i,2*j:2*j+2]; b=post[i,2*j:2*j+2]
        na=np.linalg.norm(a)+1e-12; nb=np.linalg.norm(b)+1e-12
        f[i,0]=np.dot(a,b)/(na*nb)
        f[i,1]=(a[0]*b[1]-a[1]*b[0])/(na*nb)
    return f

def qaf_nophase(pre,post,ctx):
    f=np.zeros((len(ctx),2))
    for i,j in enumerate(ctx):
        a=pre[i,2*j:2*j+2]; b=post[i,2*j:2*j+2]
        na=np.linalg.norm(a)+1e-12; nb=np.linalg.norm(b)+1e-12
        f[i]=[np.dot(a,b)/(na*nb),np.log(nb/na)]
    return f

def qaf_nocontext(pre,post):
    allf=[]
    for j in range(M):
        a=pre[:,2*j:2*j+2]; b=post[:,2*j:2*j+2]
        na=np.linalg.norm(a,axis=1)+1e-12; nb=np.linalg.norm(b,axis=1)+1e-12
        dot=np.sum(a*b,axis=1)/(na*nb)
        wedge=(a[:,0]*b[:,1]-a[:,1]*b[:,0])/(na*nb)
        allf.append(np.c_[dot,wedge])
    return np.mean(np.stack(allf,axis=2),axis=2)

def relational(pre,post,ctx):
    f=np.zeros((len(ctx),5))
    for i,j in enumerate(ctx):
        a=pre[i,2*j:2*j+2]; b=post[i,2*j:2*j+2]
        na=np.linalg.norm(a)+1e-12; nb=np.linalg.norm(b)+1e-12
        dot=np.dot(a,b)/(na*nb)
        f[i]=[dot,np.log(nb/na),abs(a[0]-b[0]),abs(a[1]-b[1]),np.linalg.norm(a-b)]
    return f

def clustering_methods(X,pre,post,ctx,seed):
    out={}
```

```
    raw=StandardScaler().fit_transform(X)
    t=time.perf_counter()
    out["Raw-KMeans"]=(KMeans(K,n_init=12,random_state=seed).fit_predict(raw),(time.perf_counter()-t)*1000)
    t=time.perf_counter()
    pca=PCA(n_components=min(12,raw.shape[1]),svd_solver="randomized",random_state=seed).fit_transform(raw)
    out["PCA-KMeans"]=(KMeans(K,n_init=12,random_state=seed).fit_predict(pca),(time.perf_counter()-t)*1000)
    t=time.perf_counter()
    out["Spectral-
Raw"]=(SpectralClustering(K,affinity="nearest_neighbors",n_neighbors=12,assign_labels="kmeans",random_state=seed).fit_predict(raw)
,(time.perf_counter()-t)*1000)
    t=time.perf_counter()
    rr=StandardScaler().fit_transform(relational(pre,post,ctx))
    out["Context-Relational"]=(KMeans(K,n_init=15,random_state=seed).fit_predict(rr),(time.perf_counter()-t)*1000)
    t=time.perf_counter()
    q0=StandardScaler().fit_transform(qaf_nophase(pre,post,ctx))
    out["QAF-NoPhase"]=(KMeans(K,n_init=15,random_state=seed).fit_predict(q0),(time.perf_counter()-t)*1000)
    t=time.perf_counter()
    q1=StandardScaler().fit_transform(qaf_nocontext(pre,post))
    out["QAF-NoContext"]=(KMeans(K,n_init=15,random_state=seed).fit_predict(q1),(time.perf_counter()-t)*1000)
    t=time.perf_counter()
    qq=qaf(pre,post,ctx)
    out["QAF-Coherence"]=(KMeans(K,n_init=20,random_state=seed).fit_predict(qq),(time.perf_counter()-t)*1000)
    return out

def run_clustering():
    rows=[]
    families=HIDDEN+[CONTROL]
    for family in families:
        for noise in NOISE:
            for rep in range(REPS):
                seed=SEED_BASE+families.index(family)*100000+int(noise*1000)*100+rep
                X,y,ctx,pre,post=gen_latent(noise=noise,seed=seed,family=family)
                for method,(pred,rt) in clustering_methods(X,pre,post,ctx,seed).items():
                    rows.append(dict(family=family,noise=noise,rep=rep,seed=seed,method=method,
                                     ari=adjusted_rand_score(y,pred),
                                     nmi=normalized_mutual_info_score(y,pred),
                                     matched_accuracy=cluster_acc(y,pred),runtime_ms=rt))
    df=pd.DataFrame(rows); df.to_csv(RESULTS/"clustering_raw.csv",index=False); return df

def anomaly_methods(X,pre,post,ctx,seed):
    out={}; raw=StandardScaler().fit_transform(X)
    t=time.perf_counter(); m=IsolationForest(n_estimators=150,contamination=CONTAM,random_state=seed).fit(raw)
    out["IsolationForest-Raw"]=(-m.score_samples(raw),(time.perf_counter()-t)*1000)
    t=time.perf_counter(); m=LocalOutlierFactor(n_neighbors=30); m.fit_predict(raw)
    out["LOF-Raw"]=(-m.negative_outlier_factor_,(time.perf_counter()-t)*1000)
    t=time.perf_counter(); m=OneClassSVM(kernel="rbf",nu=CONTAM,gamma="scale").fit(raw)
    out["OneClassSVM-Raw"]=(-m.score_samples(raw),(time.perf_counter()-t)*1000)
    t=time.perf_counter(); rr=StandardScaler().fit_transform(relational(pre,post,ctx))
    m=IsolationForest(n_estimators=150,contamination=CONTAM,random_state=seed).fit(rr)
    out["IsolationForest-Relational"]=(-m.score_samples(rr),(time.perf_counter()-t)*1000)
    t=time.perf_counter(); qq=qaf(pre,post,ctx); m=GaussianMixture(4,n_init=4,random_state=seed).fit(qq)
    out["QAF-Coherence-GMM"]=(-m.score_samples(qq),(time.perf_counter()-t)*1000)
    return out

def run_anomaly():
    rows=[]
    for noise in NOISE:
        for rep in range(ANOM_REPS):
            seed=SEED_BASE+500000+int(noise*1000)*100+rep
            X,y,ctx,pre,post=gen_anom(noise=noise,seed=seed)
            for method,(score,rt) in anomaly_methods(X,pre,post,ctx,seed).items():
                rows.append(dict(noise=noise,rep=rep,seed=seed,method=method,
                                 roc_auc=roc_auc_score(y,score),
                                 average_precision=average_precision_score(y,score),
                                 runtime_ms=rt))
    df=pd.DataFrame(rows); df.to_csv(RESULTS/"anomaly_raw.csv",index=False); return df

def run_irrelevant():
    rows=[]
    for irr in [0,16,64,128]:
        for rep in range(4):
            seed=SEED_BASE+700000+irr*10+rep
            X,y,ctx,pre,post=gen_latent(n=700,noise=.30,irr=irr,seed=seed,family="phase4")
            raw=StandardScaler().fit_transform(X)
            methods={
                "Raw-KMeans":KMeans(K,n_init=10,random_state=seed).fit_predict(raw),
                "Context-
Relational":KMeans(K,n_init=12,random_state=seed).fit_predict(StandardScaler().fit_transform(relational(pre,post,ctx))),
                "QAF-Coherence":KMeans(K,n_init=15,random_state=seed).fit_predict(qaf(pre,post,ctx))
            }
            for method,p in methods.items():
                rows.append(dict(irrelevant_dims=irr,rep=rep,method=method,ari=adjusted_rand_score(y,p)))
```

```
    df=pd.DataFrame(rows); df.to_csv(RESULTS/"irrelevant_stress.csv",index=False); return df

def run_scaling():
    rows=[]
    for n in [1000,2500,5000,10000]:
        for rep in range(2):
            seed=SEED_BASE+900000+n+rep
            X,y,ctx,pre,post=gen_latent(n=n,noise=.25,seed=seed,family="phase4")
            t=time.perf_counter(); raw=StandardScaler().fit_transform(X); KMeans(K,n_init=8,random_state=seed).fit_predict(raw)
            rows.append(dict(n_samples=n,rep=rep,method="Raw-KMeans",runtime_ms=(time.perf_counter()-t)*1000))
            t=time.perf_counter(); rr=StandardScaler().fit_transform(relational(pre,post,ctx));
KMeans(K,n_init=8,random_state=seed).fit_predict(rr)
            rows.append(dict(n_samples=n,rep=rep,method="Context-Relational",runtime_ms=(time.perf_counter()-t)*1000))
            t=time.perf_counter(); qq=qaf(pre,post,ctx); KMeans(K,n_init=8,random_state=seed).fit_predict(qq)
            rows.append(dict(n_samples=n,rep=rep,method="QAF-Coherence",runtime_ms=(time.perf_counter()-t)*1000))
    df=pd.DataFrame(rows); df.to_csv(RESULTS/"scaling_runtime.csv",index=False); return df

def stats(cluster,anom):
    hidden=cluster[cluster.family.isin(HIDDEN)].copy()
    hsum=hidden.groupby("method")[["ari","nmi","matched_accuracy","runtime_ms"]].mean().sort_values("ari",ascending=False)
    hsum.to_csv(RESULTS/"hidden_pattern_summary.csv")
    csum=cluster[cluster.family.eq(CONTROL)].groupby("method")
[["ari","nmi","matched_accuracy","runtime_ms"]].mean().sort_values("ari",ascending=False)
    csum.to_csv(RESULTS/"visible_control_summary.csv")
    asum=anom.groupby("method")[["roc_auc","average_precision","runtime_ms"]].mean().sort_values("roc_auc",ascending=False)
    asum.to_csv(RESULTS/"anomaly_summary.csv")

    p=hidden.pivot_table(index=["family","noise","rep"],columns="method",values="ari")
    d=p["QAF-Coherence"]-p["Context-Relational"]
    w=wilcoxon(p["QAF-Coherence"],p["Context-Relational"],alternative="greater")
    se=d.std(ddof=1)/np.sqrt(len(d))

    ap=anom.pivot_table(index=["noise","rep"],columns="method",values="roc_auc")
    da=ap["QAF-Coherence-GMM"]-ap["IsolationForest-Relational"]
    wa=wilcoxon(ap["QAF-Coherence-GMM"],ap["IsolationForest-Relational"],alternative="greater")

    result=pd.DataFrame([{
        "hidden_pairs":len(d),
        "qaf_mean_ari":p["QAF-Coherence"].mean(),
        "relational_mean_ari":p["Context-Relational"].mean(),
        "absolute_ari_advantage":d.mean(),
        "relative_ari_improvement_pct":(p["QAF-Coherence"].mean()-p["Context-Relational"].mean())/max(abs(p["Context-
Relational"].mean()),1e-12)*100,
        "qaf_hidden_win_rate_pct":(d>0).mean()*100,
        "ari_ci95_low":d.mean()-1.96*se,
        "ari_ci95_high":d.mean()+1.96*se,
        "ari_wilcoxon_p":float(w.pvalue),
        "qaf_anomaly_auc":ap["QAF-Coherence-GMM"].mean(),
        "relational_anomaly_auc":ap["IsolationForest-Relational"].mean(),
        "anomaly_auc_advantage":da.mean(),
        "qaf_anomaly_win_rate_pct":(da>0).mean()*100,
        "anomaly_wilcoxon_p":float(wa.pvalue)
    }])
    result.to_csv(RESULTS/"primary_statistics.csv",index=False)
    return hsum,csum,asum,result

def figures(cluster,anom,irr,scaling):
    X,y,ctx,pre,post=gen_latent(n=1000,noise=.25,seed=SEED_BASE+123,family="phase4")
    raw=StandardScaler().fit_transform(X); qq=qaf(pre,post,ctx)

    fig,ax=plt.subplots(figsize=(11,6)); ax.axis("off")
    boxes=[(.10,.70,"Pre/pos estado\n+ contexto"),(.33,.70,"Selección del\nsubespacio relevante"),(.56,.70,"Coherencia QAF\nRe:
alineación\nIm: orientación"),(.80,.70,"Clustering / densidad\nsin etiquetas"),(.33,.28,"Regímenes latentes"),(.70,.28,"Anomalías
de coherencia")]
    for x0,y0,l in boxes:
ax.text(x0,y0,l,ha="center",va="center",transform=ax.transAxes,bbox=dict(boxstyle="round,pad=.5",fill=False))
    for p1,p2 in [((.16,.70),(.26,.70)),((.40,.70),(.49,.70)),((.63,.70),(.73,.70)),((.56,.62),(.37,.36)),((.68,.62),(.69,.36))]:
        ax.annotate("",xy=p2,xytext=p1,xycoords=ax.transAxes,arrowprops=dict(arrowstyle="->"))
    ax.set_title("Figura 1. Arquitectura experimental de patrones ocultos"); savefig(fig,"fig01_architecture.png")

    fig,ax=plt.subplots(figsize=(7,6)); ax.scatter(raw[:,0],raw[:,1],c=y,s=11); ax.set_xlabel("Observable 1");
ax.set_ylabel("Observable 2"); ax.set_title("Figura 2. Espacio observable: regímenes superpuestos");
savefig(fig,"fig02_raw_overlap.png")
    fig,ax=plt.subplots(figsize=(7,6)); ax.scatter(qq[:,0],qq[:,1],c=y,s=11); ax.set_xlabel("Coherencia real");
ax.set_ylabel("Coherencia antisimétrica"); ax.set_title("Figura 3. Espacio QAF: regímenes ocultos separados");
savefig(fig,"fig03_qaf_space.png")

    hidden=cluster[cluster.family.isin(HIDDEN)]
    g=hidden.groupby("method").ari.mean().sort_values()
    fig,ax=plt.subplots(figsize=(10,6)); ax.barh(g.index,g.values); ax.set_xlabel("ARI"); ax.set_title("Figura 4. Recuperación
media de patrones ocultos"); savefig(fig,"fig04_hidden_ari.png")
    g=hidden.groupby("method").nmi.mean().sort_values()
```

```
    fig,ax=plt.subplots(figsize=(10,6)); ax.barh(g.index,g.values); ax.set_xlabel("NMI"); ax.set_title("Figura 5. Información
recuperada del régimen latente"); savefig(fig,"fig05_hidden_nmi.png")

    focus=["Raw-KMeans","Context-Relational","QAF-NoPhase","QAF-Coherence"]
    g=hidden[hidden.method.isin(focus)].groupby(["noise","method"]).ari.mean().reset_index()
    fig,ax=plt.subplots(figsize=(9,6))
    for m in focus:
        s=g[g.method.eq(m)]; ax.plot(s.noise,s.ari,marker="o",label=m)
    ax.set_xlabel("Ruido"); ax.set_ylabel("ARI"); ax.set_title("Figura 6. Robustez frente a ruido"); ax.legend();
savefig(fig,"fig06_noise.png")

    g=irr.groupby(["irrelevant_dims","method"]).ari.mean().reset_index()
    fig,ax=plt.subplots(figsize=(9,6))
    for m in ["Raw-KMeans","Context-Relational","QAF-Coherence"]:
        s=g[g.method.eq(m)]; ax.plot(s.irrelevant_dims,s.ari,marker="o",label=m)
    ax.set_xlabel("Dimensiones irrelevantes"); ax.set_ylabel("ARI"); ax.set_title("Figura 7. Robustez a variables irrelevantes");
ax.legend(); savefig(fig,"fig07_irrelevant.png")

    g=anom.groupby(["noise","method"]).roc_auc.mean().reset_index()
    fig,ax=plt.subplots(figsize=(9,6))
    for m in ["IsolationForest-Raw","OneClassSVM-Raw","IsolationForest-Relational","QAF-Coherence-GMM"]:
        s=g[g.method.eq(m)]; ax.plot(s.noise,s.roc_auc,marker="o",label=m)
    ax.set_xlabel("Ruido"); ax.set_ylabel("ROC-AUC"); ax.set_title("Figura 8. Detección de anomalías de coherencia"); ax.legend();
savefig(fig,"fig08_anomaly_auc.png")

    g=scaling.groupby(["n_samples","method"]).runtime_ms.mean().reset_index()
    fig,ax=plt.subplots(figsize=(9,6))
    for m in ["Raw-KMeans","Context-Relational","QAF-Coherence"]:
        s=g[g.method.eq(m)]; ax.plot(s.n_samples,s.runtime_ms,marker="o",label=m)
    ax.set_xlabel("Muestras"); ax.set_ylabel("Tiempo (ms)"); ax.set_title("Figura 9. Escalabilidad temporal"); ax.legend();
savefig(fig,"fig09_scaling.png")

    p=hidden.pivot_table(index=["family","noise","rep"],columns="method",values="ari").reset_index()
    p["adv"]=p["QAF-Coherence"]-p["Context-Relational"]
    hm=p.groupby(["family","noise"]).adv.mean().unstack("noise")
    fig,ax=plt.subplots(figsize=(8,5.5)); im=ax.imshow(hm.values,aspect="auto")
    ax.set_xticks(range(len(hm.columns))); ax.set_xticklabels([f"{v:.2f}" for v in hm.columns])
    ax.set_yticks(range(len(hm.index))); ax.set_yticklabels(hm.index)
    ax.set_xlabel("Ruido"); ax.set_ylabel("Familia"); ax.set_title("Figura 10. Ventaja QAF sobre baseline relacional (Δ ARI)")
    for i in range(hm.shape[0]):
        for j in range(hm.shape[1]): ax.text(j,i,f"{hm.iloc[i,j]:.2f}",ha="center",va="center")
    fig.colorbar(im,ax=ax,label="Δ ARI"); savefig(fig,"fig10_heatmap.png")

def main():
    t=time.perf_counter()
    cluster=run_clustering()
    anom=run_anomaly()
    irr=run_irrelevant()
    scaling=run_scaling()
    h,c,a,s=stats(cluster,anom)
    figures(cluster,anom,irr,scaling)
    print("Benchmark completed")
    print(f"Clustering executions: {len(cluster)}")
    print(f"Anomaly executions: {len(anom)}")
    print(f"Wall time: {time.perf_counter()-t:.2f} s")
    print("\nHidden pattern summary:")
    print(h.round(4).to_string())
    print("\nVisible control:")
    print(c.round(4).to_string())
    print("\nHidden anomaly summary:")
    print(a.round(4).to_string())
    print("\nPrimary statistics:")
    print(s.T.to_string(header=False))
    print("\nEnvironment:",platform.platform(),"Python",sys.version.split()[0])

if __name__=="__main__":
    main()
```